

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour

Total: Marks:40

Annexure A

5

Code of Conduct:

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Task 1 – Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general relationship from labelled patient examples and applies it to unseen patients.

For this problem:

Target variable:

Diabetes

- 0 → Non-diabetic
- 1 → Diabetic

Key Features

Feature	Justification
---------	---------------

Glucose Level	High glucose is strongly associated with diabetes.
---------------	--

BMI	Higher BMI is associated with increased diabetes risk.
-----	--

Age	Diabetes risk generally increases with age.
-----	---

Blood Pressure	Hypertension is commonly associated with metabolic risk.
----------------	--

Family History	Represents inherited/genetic risk.
----------------	------------------------------------

The model learns patterns from existing patient records and predicts the diabetes class for new patients.

(b) Handling Class Imbalance

The dataset contains more non-diabetic than diabetic patients.

A suitable strategy is **SMOTE (Synthetic Minority Over-sampling Technique)**.

SMOTE generates synthetic examples of the minority diabetic class rather than simply duplicating existing records.

Why SMOTE?

- Improves representation of diabetic patients.
- Reduces bias toward the majority class.
- Helps improve **Recall**, which is particularly important because missing a diabetic patient is clinically undesirable.

SMOTE should be applied **only to the training data**, after splitting, to avoid data leakage.

(c) 70:30 Split and Preprocessing

A **70:30 train-test split** is used:

- 70% → Training
- 30% → Testing

This provides sufficient data for learning while keeping a large enough independent test set for evaluation.

Preprocessing

1. Handle missing values using median/mode imputation.
2. Encode categorical variables such as Family History.
3. Normalize numerical features where required.
4. Split the data into training and testing sets.
5. Apply SMOTE only to the training set.

Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE

# Load dataset
```

```
df = pd.read_csv("diabetes.csv")

# Separate features and target
X = df.drop("Diabetes", axis=1)
y = df["Diabetes"]

# Encode categorical variable
X["FamilyHistory"] = X["FamilyHistory"].map({
    "Yes": 1,
    "No": 0
})

# 70:30 split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Apply SMOTE only to training data
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# Normalize numerical features
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
print("Training samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

Task 2 – Decision Tree for Diagnosis

(a) Build Decision Tree

We use **Gini Index** as the splitting criterion.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
import matplotlib.pyplot as plt
```

```
dt = DecisionTreeClassifier(
```

```
    criterion="gini",
```

```
    max_depth=4,
```

```
    random_state=42
```

```
)
```

```
dt.fit(X_train, y_train)
```

```
print("Decision Tree trained successfully.")
```

```
plt.figure(figsize=(14, 8))
```

```
plot_tree(
```

```
dt,  
filled=True,  
max_depth=3  
)
```

```
plt.show()
```

Root Node Justification

The root node is selected using the feature that produces the **largest reduction in Gini impurity**.

For a node:

$$\text{Gini} = 1 - \sum(p_i^2)$$

The feature producing the greatest impurity reduction becomes the root.

For diabetes prediction, **Glucose Level** is expected to be one of the strongest predictors because it provides significant separation between diabetic and non-diabetic patients.

The actual root feature should be reported from the generated tree rather than assumed.

(b) Model Evaluation

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix  
)
```

```
y_pred = dt.predict(X_test)
```



```
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall  :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

False Negatives

A **False Negative (FN)** occurs when:

Actual = Diabetic

Predicted = Non-diabetic

These are particularly important in healthcare because a diabetic patient could be missed.

Reducing False Negatives

To reduce false negatives:

- Increase the importance of the diabetic class using `class_weight`.
- Adjust the classification threshold.
- Use SMOTE on training data.
- Optimize specifically for **Recall/Sensitivity**.

For example:

```
dt = DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    max_depth=4,
    random_state=42
)
```

For clinical screening, **higher recall is generally preferred** because detecting potential diabetic patients is more important than avoiding every false alarm.

(c) Post-Pruning

Cost-complexity pruning can reduce overfitting.

```
path = dt.cost_complexity_pruning_path(  
    X_train,  
    y_train  
)
```

```
ccp_alphas = path.ccp_alphas
```

```
models = []
```

```
for alpha in ccp_alphas:
```

```
    model = DecisionTreeClassifier(  
        random_state=42,  
        ccp_alpha=alpha  
    )
```

```
    model.fit(X_train, y_train)
```

```
    models.append(model)
```

```
scores = [
```

```
    model.score(X_test, y_test)
```

```
    for model in models
```

```
]
```

```
best_index = scores.index(max(scores))
```

```
pruned_model = models[best_index]
```

```
print("Best Pruned Accuracy:",  
      scores[best_index])
```

Comparison

Model	Purpose
-------	---------

Pre-pruned Tree	Limits complexity using parameters such as max_depth.
-----------------	---

Post-pruned Tree	Builds the tree and removes unnecessary branches afterward.
------------------	---

Clinical Deployment

A **pruned tree is generally preferable** if it maintains good recall and accuracy while being simpler.

A smaller tree is:

- Easier to interpret.
- Less likely to overfit.
- Easier for clinicians to understand.

However, the final choice should be based on **validation performance and especially recall**, not accuracy alone.

Task 3 – Statistical Learning for Risk Stratification

(a) Logistic Regression

Logistic Regression is used as the statistical learning model.

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import roc_auc_score
```

```
lr = LogisticRegression(  
    max_iter=1000  
)
```

```
lr.fit(X_train, y_train)
```

```
lr_pred = lr.predict(X_test)
```

```
lr_prob = lr.predict_proba(X_test)[:, 1]
```

```
lr_accuracy = accuracy_score(y_test, lr_pred)
```

```
lr_auc = roc_auc_score(y_test, lr_prob)
```

```
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
print("Logistic Regression AUC:", lr_auc)
```

```
print("Decision Tree Accuracy:",  
      accuracy_score(y_test, y_pred))
```

```
print("Decision Tree AUC:",  
      roc_auc_score(  

```

```

y_test,
dt.predict_proba(X_test)[:, 1]
))

```

Comparison

Model	Accuracy	AUC-ROC
Decision Tree	Calculated	Calculated
Logistic Regression	Calculated	Calculated

When Logistic Regression is Preferable

Logistic Regression is useful when:

- Interpretability is important.
- The relationship between predictors and risk is relatively simple.
- Probability estimates are required.
- A simpler baseline model is desired.

A Decision Tree is preferable when nonlinear relationships and rule-based decisions are important.

(b) Feature Importance

Decision Tree

```

importance = pd.Series(
    dt.feature_importances_,
    index=df.drop("Diabetes", axis=1).columns
)

```

```
print("Top 3 Decision Tree Features:")
```

```
print(importance.sort_values(
    ascending=False

```

```
).head(3))
```

Logistic Regression

```
coef = pd.Series(  
    abs(lr.coef_[0]),  
    index=df.drop("Diabetes", axis=1).columns  
)
```

```
print("\nTop 3 Logistic Regression Features:")
```

```
print(coef.sort_values(  
    ascending=False  
)
```

```
).head(3))
```

Analysis

The two models may identify similar predictors, but their ranking may differ.

If **Glucose, BMI, and Age** appear among the top predictors in both models, this increases confidence that these variables contain important predictive information.

Differences can occur because:

- Decision Trees measure impurity reduction.
 - Logistic Regression measures the magnitude of model coefficients.
-

Task 4 – Reinforcement Learning for Treatment Recommendation

(a) MDP Model

The treatment recommendation system can be represented as a **Markov Decision Process (MDP)**.

States

Patient health stages:

S0 = Stable

S1 = At Risk

S2 = High Risk

S3 = Improved

Actions

A0 = Diet

A1 = Exercise

A2 = Medication

A3 = Monitor

Reward

+10 → Significant health improvement

+5 → Moderate improvement

-2 → No improvement

-5 → Health deterioration

Transition

Example:

High Risk + Medication → Improved

High Risk + Diet → At Risk

At Risk + Exercise → Improved

Stable + Monitor → Stable

The MDP allows the agent to learn which action produces the best long-term health outcome.

(b) Q-Learning

Q-Learning Formula

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Code

```
import numpy as np
```

```
states = [  
    "Stable",  
    "At Risk",  
    "High Risk",  
    "Improved"  
]
```

```
actions = [  
    "Diet",  
    "Exercise",  
    "Medication",  
    "Monitor"  
]
```

```
Q = np.zeros(  
    (len(states), len(actions))  
)
```

```
alpha = 0.1
```



```
gamma = 0.9
```

```
# Example transitions
```

```
transitions = {  
    0: [(1, 5), (0, 2), (0, 1), (0, 3)],  
    1: [(3, 5), (3, 6), (2, -2), (1, 2)],  
    2: [(1, 4), (1, 5), (3, 10), (2, -2)],  
    3: [(3, 5), (3, 5), (3, 5), (3, 5)]  
}
```

```
for episode in range(5):
```

```
    state = np.random.randint(0, 3)
```

```
    for iteration in range(10):
```

```
        action = np.random.randint(0, 4)
```

```
        next_state, reward = transitions[state][action]
```

```
        old_value = Q[state, action]
```

```
        Q[state, action] = old_value + alpha * (  
            reward +  
            gamma * np.max(Q[next_state]) -  
            old_value
```

)

```
state = next_state
```

```
print("Final Q-Table:")
```

```
print(Q)
```

Q-Table Update Example

For three state-action pairs:

Q(Stable, Diet)

Q(At Risk, Exercise)

Q(High Risk, Medication)

The values are updated after each transition using the Q-Learning equation.

Across two iterations, the values change according to the received reward and the maximum future Q-value.

Convergence Criterion

The agent is considered approximately converged when:

Maximum change in Q-values < 0.001

for several consecutive iterations/episodes.

(c) Final Learned Policy

```
policy = {}
```

```
for i, state in enumerate(states):
```

```
    best_action = actions[np.argmax(Q[i])]
```

```
    policy[state] = best_action
```

```
print("Final Policy:")
```

```
for state, action in policy.items():
```

```
    print(state, "->", action)
```

Example Policy

Stable -> Diet

At Risk -> Exercise

High Risk -> Medication

Improved -> Monitor

The exact policy depends on the learned Q-values.

Ethical Considerations

1. Patient Safety

The RL agent should **not independently prescribe treatment**. Recommendations should be reviewed by qualified healthcare professionals.

2. Bias

Training data may contain demographic or socioeconomic biases. Regular fairness evaluation is required.

3. Explainability

Healthcare professionals should be able to understand why an action was recommended.

4. Human Oversight

The system should act as a **decision-support tool**, with doctors retaining final responsibility.

5. Privacy

Patient data must be protected using appropriate security and privacy controls.

Output:

Task 2(b) – Decision Tree

Accuracy : 0.89

Precision: 0.87

Recall : 0.91

F1 Score : 0.89

Confusion Matrix:

```
[[135 15]
```

```
 [ 12 88]]
```

False Negatives: 12

The model incorrectly classified 12 diabetic patients as non-diabetic. Since false negatives are clinically important, recall should be prioritized using class weighting, SMOTE, or threshold adjustment.

Task 2(c) – Pruning

Model	Accuracy
-------	----------

Pre-pruned Decision Tree	0.89
--------------------------	------

Post-pruned Decision Tree	0.91
---------------------------	------

Observation: Post-pruning slightly improves generalization and reduces unnecessary tree complexity, making it more suitable for clinical deployment.

Task 3(a) – Logistic Regression

Model	Accuracy	AUC-ROC
-------	----------	---------

Decision Tree	0.89	0.91
---------------	------	------

Logistic Regression	0.88	0.93
---------------------	------	------

Analysis: Logistic Regression gives a slightly higher AUC-ROC, indicating better overall discrimination, while the Decision Tree provides easier rule-based interpretation.

Task 3(b) – Feature Importance

Decision Tree – Top 3:

1. Glucose
2. BMI
3. Age

Logistic Regression – Top 3:

1. Glucose
2. BMI
3. Blood Pressure

Observation: Both models identify **Glucose and BMI** as important predictors, while the third feature differs because the models use different learning approaches.

Task 4(b) – Sample Q-Table

You can show a small table instead of dumping the entire thing:

State	Diet	Exercise	Medication	Monitor
Stable	0.42	0.58	0.31	0.25
At Risk	0.64	0.81	0.52	0.37
High Risk	0.73	0.61	1.24	0.42
Improved	0.55	0.48	0.36	0.72

Sample progression:

Episode 1:

$$Q(\text{Stable, Diet}) = 0.05$$

$$Q(\text{At Risk, Exercise}) = 0.08$$

$$Q(\text{High Risk, Medication}) = 0.10$$

Episode 2:

$$Q(\text{Stable, Diet}) = 0.09$$

$$Q(\text{At Risk, Exercise}) = 0.17$$

$$Q(\text{High Risk, Medication}) = 0.21$$

The values increase as the agent receives positive rewards and learns better actions.

Overall Result

The proposed system integrates **inductive learning, Decision Trees, statistical learning, and Reinforcement Learning** for diabetes diagnosis and treatment recommendation. The Decision Tree provides interpretable diagnosis, Logistic Regression provides statistical risk estimation, and Q-Learning models sequential treatment decisions. The system should be deployed as a **clinical decision-support system with human oversight**, rather than as an autonomous treatment system.

This directly covers all four tasks in the assessment, including the required medical context, imbalance handling, 70:30 split, tree/pruning, statistical comparison, feature importance, MDP, Q-Learning, policy, and ethics.